

E04 · 얼마나 담기나 — 기억용량

앞 권은 질문 하나를 열어 둔 채 끝났다. 오프라인 라운드마다 가중치에 새지식을 밀어 넣는다면, **가중치에는 애초에 얼마나 들어가는가**. 이 권은 본문 ch09(기억 용량)와 이 책의 회계 실험 X2를 수식과 증명 없이 다시설명한다.

답의 한가운데에는 이 라인에서 가장 자주 인용되는 수 하나가 있다 — **파라미터 하나당 3.64 bits**. 이 권의 앞 절반은 그수가 무엇을 재서 나왔는지에 쓰고, 뒤 절반은 그것이 **천장이 아니라바닥**이라는 사실과, 이 책이 초판에서 그 방향을 거꾸로 써서 결론 하나를 철회한 이야기에 쓴다. 자기 오류를 밝히는 것이 이 책의 방식이다.

1. 이 권이 쓰는 단위는 bits 다

같은 “용량”이라는 단어가 이 라인에서 최소 세 가지 다른 양을 가리킨다. 그래서 첫 문단이 단위를 고정한다.

기호 풀이 Θ (slow weights) = 배포되는 가중치 그 자체. .safetensors로 굽는 것. W (fast weights) = 질의를 처리하는 동안 움직이는 고정 크기 상태. E (external store) = 모델 밖의 텍스트·벡터·그래프 저장소. C_{cap} = **상태 용량**. 이 라인의 비용 4종 중 하나이며, 나머지 셋은 B_s (오프라인 라운드의 계산 예산) · L_w (질의 지연 기여) · ρ (라운드당 성능 손실) 다. $|\Theta|$ = 파라미터의 **개수**.

Neural Memory 판이 켜 용량은 W 의 용량이었고 단위가 **key-value 쌍의 개수**였다. 이 권이 재는 것은 Θ 의 용량이고 단위는 **bits**다. 두 수는 이름만 같고 서로 환산되지 않는다.

기호 풀이 **bit** = 예/아니오 하나를 가르는 정보량. n bits는 2^n 갈래를 구분한다. 파라미터 하나가 3.64 bits를 담는다는 것은 그 파라미터가 대략 $2^{3.64} \approx 12.5$ 갈래의 선택을 기억하고 있다는 뜻이다. 사양이 아니라 실측 결과의 단위다.

여기서 곧바로 어긋나는 감각 하나를 바로잡아야 한다. bf16으로 저장한 파라미터 하나는 **16 bits의 자리**를 차지한다. 그 자리에 실제로담기는 데이터셋 정보가 3.64 bits라는 것이 이 권의 주제다. 자리와 내용물은 다른 수이며, 둘의 비가 이 권 후반부의 결론을 전부만 들어낸다.

자세한 것은 ch09 도입부.

2. 무엇을 셀 것인가 — 외운 것과 아는 것을가른다

모델이 “프랑스의 수도는 파리”를 맞혔다. 이것은 훈련 집합의 그 문장을 담고 있어서인가, 세상에 대한 일반적 압축이 그 답을 함의해서인가. 후자까지 세면 용량은 무한대로 나온다. 그래서 재기 전에 **분리**가먼저다.

How much do language models memorize(arXiv:2505.24832)는 이 분리를 **참조 모델로 조건화**해서 수행한다.

기호 풀이 Θ_{ref} = 참조 모델. 같은 데이터셋을 보지않은, 비교 기준이 되는 다른 모델. mem_U = **unintended memorization**. 참조 모델이 이미 아는 것을 먼저 빼고남은, 대상 모델만이 아는 양.

이 정의가 말하는 것. 순서가 곧 장치다. 참조 모델이 이미 아는것을 먼저 빼고 시작하므로, 남는 양은 “이 데이터셋을 봤기 때문에 생긴” 정보다. 일반화는 직접 재지 않고 전체에서 이것을 뺀 잔차로만정의된다.

비유 같은 파일을 두 코덱으로 압축했을 때의 코드 길이 차이다. 대상 모델이라는코덱이 참조 코덱보다 이 문자열을 얼마나 더 짧게 쓰는가 — 그 차이가대상 모델이 이 문자열에 대해 따로 들고 있는 bits 다. — **비유가깨지는 곳**: 코덱 비교에서는 기준 코덱이 고정되어 있지만, 여기서는 참조모델을 무엇으로 잡느냐에 따라 값이 움직인다. 논문은 참조 모델 규약을최소 세 가지로 쓰고 (더 큰 모델 / 같은 크기를 최대 데이터로학습한 모델 / 최저 손실 oracle), **어느 선택에대해서도 민감도 분석을 보고하지 않는다**. 즉 mem_U 는 모델하나의 성질이 아니라 **모델 쌍의 성질**이다.

시스템 모델링 관점 이 정의가 서빙 쪽에 남기는 것은 검증 규칙이다. 오프라인 라운드가 지식을실제로 심었는지 확인하려고 **프롬프트를 넣고 출력이 맞는지 보는방식은 양방향으로 틀린다**. 논문은 membership inference 점수가 0.97 인데 32-token prefix 추출률이 0 인 사례를 보고하고, 반대방향에 대해서도 못을 박는다 — “언어모델은 거의 아무 문자열이나출력하도록 유도될 수 있으므로, 모델이 무언가를 출력한다는 사실 자체가암기의 증거는 아니다” [lm-memorization-capacity § 1]. 담고 있으면서뱉지 않을 수 있고, 답지 않은 채로 뱉을 수 있다. 옳은 게이트는 가능성비다— 같은 문자열에 대해 두 모델의 forward pass 를 각각 한 번 돌리고 뺀다. **추가 학습도, 갱신 기제도 필요 없는 사후 측정이고, 배치로 돌릴 수있다**. 이 라인의 Θ -경로 논문 중 이 게이트를 쓰는 논문은 없다.

자세한 것은 ch09 § 09.1.

3. 재는 방법 — 외울 수밖에 없는 데이터를준다

정의는 측정이 아니다. 측정으로 바꾸려면 데이터의 정보량을 **정확히**아는 자료가 필요하다. 논문의 수법은 일반화가 원리적으로 불가능한데이터를 쓰는 것이다: 어휘 2048 개에서 토큰을 균등·독립으로뽑아 길이 64 의 문자열을 만든다.

토큰이 서로 독립이므로 앞을 보고 뒤를 예측할 방법이 없다. 규칙이 없으니외우는 것 말고는 맞힐 길이 없고, 그래서 “외운 것” 과 “안 것” 의 분리 문제자체가 사라진다. 정보량도 닫힌 형태로 나온다 — 어휘 2048 개면 토큰 하나가 11 bits, 길이 64 면 문자열 하나가 704 bits 다.

이제 데이터셋 크기만 키우면서 포화까지 학습시킨다. 암기량은 처음에데이터와 함께 늘다가 어느 지점에서 **평평해진다**. 그 평평해진높이가 그 모델이 담은 총량이고, 그것을 파라미터 개수로 나눈 기율기가 **bits per parameter(bpp)** 다. GPT 계열 half precision 에서 3.64 로 보고된다 [lm-memorization-capacity Fig. 6 캡션].

수를 인용할 때 함께 옮겨야 할 것이 측정 조건이다. 이 표가 곧 그 수의사용 범위다.

표 E04-1. 3.64 bits per parameter 가 측정된 조건 [lm-memorization-capacity § 3.2].

조건	값
데이터	균등 · 독립 무작위 토큰 문자열 (실제 텍스트 아님)
아키텍처	GPT-2, from scratch 학습
규모	$8.04 \times 10^4 \sim 6.86 \times 10^6$ 파라미터
학습	10^6 steps, batch 2048, Adam 하나만 , A100 1장, bfloat16, 시드 5 개
시퀀스	길이 64 tokens, 어휘 2048

주의 표에 없는 것이 표에 있는 것만큼 중요하다. 텍스트에 대한 bpp 값은 논문 어디에도 없다. adapter 도, 저랭크 갱신도, 일부 파라미터만 학습시키는 방식도 한 번도 측정되지 않았다. 표의 모든모델은 전 파라미터가 자유로운 from-scratch 학습이다. 그리고 이 상수는 6.86×10^6 파라미터 위에서 재측정된 적이 없는데, 같은 논문의다른 법칙은 이 값을 15억 파라미터 모델에 그대로 대입한다 — 약 227 배외삽이다.

자세한 것은 ch09 § 09.2.

4. 3.64는 하나의 수가 아니라밴드다

논문을 자기 헤드라인 상수를 한 값으로 쓰지 않는다. 초록은 3.6, 서론은 “3.5와 4 사이”, 결과 절은 “3.5와 3.6 사이”, 그림 캡션은 3.64, 부록은 3.642, 표의 평균은 bf16 3.51 / fp32 3.83 이다. 표의 설정별 값은 bf16 에서 2.86 – 3.94, fp32 에서 3.02 – 4.23 으로 흩어지고, 표에는 “Mean (± 0.1)” 이라고 적혀 있다.

바이트로 읽을 때 걸리는 단위 오류도 하나 있다. 그림 하나가 6.86×10^6 파라미터 모델의 용량을 “23.9 MB” 로 인쇄하는데, 같은모델에 대해 표가 주는 값은 2.51×10^7 bits = 3.14 MB 다. bit 수를 2^{20} 으로 나눈 값에 MB 를 붙인 것이다. **그그림에서 저장 바이트를 읽으면 8 배 과다 산정한다.**

핵심 존재하는 것은 하나의 상수가 아니라 **약 ± 0.5 폭의 밴드**이고, 중심값은 논문의 어느 문장을 인용하느냐에 따라 달라진다. 이 권도 계산에는 3.64 를 쓰지만, 결론은 언제나 자릿수와 순서로만 말한다. 3.64 를 소수 둘째 자리까지 믿는 서술은 이 논문이 뒷받침하지않는다.

자세한 것은 ch09 § 09.2.1.

5. 바닥인가 천장인가 — 이 책이 뒤집어 쓴이야기

여기가 이 권의 중심이다.

앞 절의 측정 절차를 다시 본다. 데이터를 키우며 gradient descent 로 학습시켰더니 곡선이 파라미터당 3.64 bits 에서 평평해졌다. 이때 확인된 사실은 정확히 하나다 — **이 절차로 채워 봤더니 거기까지는 들어갔다.**

논문 자신이 그 방향을 명시한다. “우리 모델은 gradient descent 로 학습되므로 전역 최적을 찾는다 는 보장이 없다. 따라서 우리는 언제나 모델 용량의 **하한**만을 측정한다”(원문: “we are only ever measuring a lower bound on model capacity”) [lm-memorization-capacity § 3.2]. 게다가 가장 큰 두 실행은 수렴하지 않았다고 같은 절에 적혀 있다.

비유 커널을 하나 짜서 돌렸더니 3 TB/s 가 나왔다. 이때 알게 된 것은 “이 하드웨어는 적어도 3 TB/s 를 낸다” 이지 “이 하드웨어의 최대는 3 TB/s 다” 가 아니다. 더 좋은 커널이 오면 값은 올라간다. 측정값은 아래에서 막혀 있고 위로는 열려 있다. 3.64 bits per parameter 도 정확히 그런 수다 — gradient descent 라는 커널이 낸 달성치다. — **비유가 깨지는 곳**: 대역폭에는 사양표라는 명시적 천장이 있어 달성률을 계산할 수 있지만, 용량에는 그런 사양표가 없다. 형식상의 천장은 “bf16 이면 파라미터당 16 bits” 뿐인데 이것은 물리적 최대치가 아니라 **저장 형식이 정하는 수다**. 실제 도달 가능한 최대치는 이 라인의 어느 논문도 재지 않았다.

핵심 “적어도 3.64 bits” 는 이렇게 읽는다 — 이 절차로는 여기까지 들어갔고, 더 들어갈 수 있는지는 **모른다**. 이 문장에서 “모른다” 를 “없다” 로 바꾸는 순간 모든 결론의 방향이 뒤집힌다.

방향이 뒤집히면 무엇이 달라지는지가 핵심이다. 하한은 Θ -경로에 **유리**한 진술이다. “적어도 이만큼은 들어간다” 는 말은 가중치에 지식을 넣는 방식을 반증하지 못한다. 반대로 이 값을 천장으로 읽으면 “가중치는 이것밖에 못 담는다” 가 되어 그 경로를 **반증**하는 진술로 변한다. 같은 숫자가 정반대의 논증을 떠받치게 된다.

표 E04-2. 3.64 bits per parameter 로 쓸 수 있는 문장과 쓸 수 없는 문장.

쓸 수 있다	쓸 수 없다
실측된 도달점은 파라미터당 약 3.6 bits 였다	파라미터 하나에는 3.64 bits 까지만 들어간다
암기량은 가중치 자체의 정보량을 넘을 수 없다	이 adapter 의 용량은 $3.64 \times$ 파라미터 수다
저장 dtype 을 내리면 저장 바이트당 밀도가 그대로 오른다	Θ 가 E 보다 바이트 효율이 낮다
용량이 차면 새로 쓰는 것이 옛것을 밀어낸다	3.64 를 근거로 Θ -경로는 규모가 안 나온다

한계 이 책의 회계 실험 X2 초판이 정확히 그 치환을 했다. 3.64 를 천장으로 놓고 Θ 와 E 의 저장 바이트당 정보 밀도를 나눠 경로간 비 1.47 / 0.73 / 0.37 (bf16 / int8 / int4) 을 냈고, 거기에 “ E 가 Θ 보다 바이트 효율이 높다” 는 결론을 붙였다. 원논문의 문장을 다시 읽고 방향이 반대임을 확인한 뒤 **그 비교를 철회**했다. 지금 그 자리에 남아 있는 판정은 “판정 불가” 다 [X2 F3_withdrawn].

왜 철회가 필요했는지를 한 줄로 옮기면 이렇다. Θ 쪽 값은 **하한**이고 E 쪽 값은 텍스트 바이트의 **액면가**다. 하한을 액면가로 나눈 비는 어느 방향으로도 결론을 주지 않는다 — 분자가 실제로 얼마나 더 큰지 모르기 때문이다. 두 경로의 밀도를 정말 비교하려면 delta 에 직접 정보를 주입해 회수율을 재는 실험이 필요하고, 그것은 회계로 대신할 수 있는 일이 아니다.

살아남은 것도 분명하다. 저장 dtype 축의 결론은 하한이든 상한이든 그대로 성립한다. 밀도의 분자가 무엇이든 분모는 저장 형식이 정하므로, bf16에서 int8, int4로 내리면 저장 바이트당 정보 밀도가 그대로 두 배씩 오른다. § 10이 그 수를 쓴다.

주의 여기에 **별개의** 제약이 하나 더 겹친다. 3.64를 LoRA delta 같은 저랭크 갱신에 곱하는 것은 방향 문제와 무관하게 **외삽**이다 — 그 실험은 저랭크 부분공간을 측정하지 않았다. 두 caveat은 서로를 대체하지 않는다. 하나는 “이 수는 바닥이다”, 다른 하나는 “이 수는 다른 설정에서 나왔다”이며, 곱을 쓸 때는 둘 다 붙는다.

자세한 것은 ch09 § 09.2.2와 § 09.7.

6. 용량은 그릇만의 성질이 아니다

앞 절의 측정 절차를 정의로 굳히면 이렇게 된다 — 어떤 **학습절차**의 용량은, 데이터를 바꿔가며 포화까지 학습시켰을 때 얻어지는 암기량의 최댓값이다. 인자가 아키텍처가 아니라 **학습 절차**라는 점이 핵심이다. 용량은 (아키텍처, 학습 알고리즘) 쌍의 성질이 그릇하나의 성질이 아니다. 논문의 모든 실험은 Adam 하나만 쓰고, 두 번째 optimizer는 시도되지 않았다.

정밀도 실험이 이 점을 가장 선명하게 보인다. bf16 파라미터는 16 bits의 자리를 차지하고 3.51 bits를 담는다 — 저장 효율 21.9%다. fp32는 32 bits를 차지하고 3.83을 담는다 — 12.0%다. 자리를 두 배로 늘렸는데 내용물은 9.1%만 늘었다. 논문의 표현으로 “정밀도를 bfloat16에서 float32로 올릴 때 추가된 모델 bits의 대부분은 원시 저장에 쓰이지 않는다” [lm-memorization-capacity § 3.2].

핵심 이 결과는 bpp를 저장 밀도로 읽는 틀과 어긋난다. 그릇의 성질이라면 bits를 두 배 주었을 때 용량도 두 배가 되어야 한다. 남은 문장은 하나다 — **가중치는 특정 내용을 담기에 매우 비효율적인 매체이고, 그 비효율의 원인은 매체가 아니라 그 매체를 채우는 절차다**. 값이 모델 형태에도 안정적이지 않다는 사실이 이를 뒷받침한다. 같은 표에서 층 1개 · 폭 256은 2.88인데 층 2개 · 폭 256은 3.66이고, 층 8개 · 폭 32는 2.86인데 층 1개 · 폭 32는 3.93이다.

절차와 무관하게 걸리는 천장은 따로 있다. 어떤 갱신 절차도 **가중치 자체가 담을 수 있는 정보량을 넘겨 쓸 수 없다**. 이것이 이 권이 상속하는 유일한 진짜 상한이며, 3.64와 달리 방향이 보장된다. 단 그 증명은 학습 표본들이 서로 독립이라고 가정한다 — 모델이 스스로 생성한 데이터처럼 표본끼리 상관성이 큰 학습 집합에는 그대로 옮길 수 없다.

실무형은 더 단순하다. 암기량은 **용량과 데이터 정보량 중 작은 쪽**이다. 즉 찰 때까지 채우고, 차면 멈춘다. 이 근사의 검증 오차는 조건을 바꿔 가며 1.7%와 1.8%였다.

여기서 두 가지가 따라 나온다. 첫째, **경계가 사전에 계산된다**. 오프라인 라운드를 돌리기 전에 쓰려는 자료의 정보량과 대상 파라미터 블록의 크기만으로 “이 블록은 이미 찼는가”를 판정할 수 있다. 논문은 그런 scheduler를 제안하지 않는다 — 이것은 이 책의 재구성이다. 둘째, **포화 후의 쓰기는 제로섬**이다. 데이터가 늘면 모델은 암기량을 표본들에 나눠 배분하고 합은 일정하게 측정된다. 반복되는 Θ -경로 라운드는 언젠가 반드시 잇는다. 이것은 설계 실패가 아니라 천장의 귀결이다.

한계 다만 이 논문은 망각의 **필연성**만 주고 **속도**는 주지않는다. 라운드라는 개념 자체가 없으므로 ρ 가 정의조차 되지 않는다. 이 절이 소유한 것은 천장이지 감쇠 곡선이 아니다. 그리고 무엇이 그릇을빨리 채우는지에 대해 불편한 관측이 하나 있다 — 희귀어를 포함한 표본이더 많이 암기되며, 가장 많이 암기된 상위 20 개 시퀀스 중 17개가일본어·중국어·히브리어 등 비영어 토큰열이다. 사용자별 오프라인 스킴이 통합하려는 자료가 정확히 이런 것이다. “잘달라붙는 내용”과 “그릇을 가장 빨리 채우는 내용”은 같은 사실이다.

자세한 것은 ch09 § 09.3.

7. LoRA 는 얼마나 답나 — 실측은 다른 단위로잔다

Θ -경로의 오프라인 라운드가 실제로 건드리는 것은 전체 가중치가아니라 작은 덧붙임이다.

기호 풀이 LoRA = 원래 가중치 행렬은 얼어붙은 채로 두고, 그 위에 더해질**저랭크 delta** 하나만 학습하는 방식. $r = \text{rank}$. delta 를 만드는 두 얇은 행렬의 공통 변. r 이 작을수록학습·저장되는 파라미터가 적다. d = 모델 폭, L_{layer} = 층 수, S = LoRA 를 붙이는 행렬들의집합.

Understanding LoRA as Knowledge Memory(arXiv:2603.01097) 가이 delta 의 용량을 직접 잔다. 다만 bits 가 아니라 **행동적 토큰수**로 잔다 — 성능이 임계 이상으로 유지되는 최대 지식 적재량을 학습 가능파라미터 수로 나눈다. § 3 의 bits per parameter 와 **형태가 같고단위가 다르다**.

보고되는 형태는 셋이다. 첫째, 용량은 rank 와 함께 커지되 rank 증가분당한계 이득은 감소한다. 둘째, 고정 rank 에서 적재량을 늘리면 어느임계까지는 유지되다가 그 지점을 넘으면 성능이 급격히 떨어진다. 셋째, **파라미터당 효율은 rank 에 대해 단조가 아니고 작은 rank 에서최대다** — 가장 효율적인 설정은 일관되게 $r = 4$ 같은 작은 rank 였다.

같은 문서라도 **학습 집합의 형식**이 용량을 좌우한다는 것도 이논문이 처음 실측한다. 원문 그대로 학습하면 LLM-judge 점수 3.187 인데, 같은 내용을 QA 40 쌍으로 바꾸면 5.893, 네 형식을 모두 섞으면 6.822 다. 내용이 아니라 형식만 바뀌서 두 배가됐다.

한계 정직하게 옮겨야 할 것이 많다. **최대 적재량의 수치가 논문 어디에도표로 없고**, 어느 rank 에 대해서도 파라미터 수가 명시되지 않는다. 즉 효율식의 값을 재현할 수 있는 점이 하나도 없고, 값은 오직 그림의 비율 축에만존재한다. 시드·표준편차·오차막대도 논문 전체에없다. 세 벤치 중 둘이 저자 제작이며, 최고 형식으로 뽑힌 QA 에 대해 저자 스스로 오염을 인정한다 — “QA 형식의 우위는 평가 과제 역시 QA 라는구조적 정렬에 부분적으로 기인할 수 있다”.

가장 중요한 것은 **부정 결과**다. 모듈을 여러 개 만들어 용량을키우는 축은 작동하지 않는다. 라우팅 오류를 완전히 제거하고 정답 모듈만병합해도 점수는 모듈 1 개에서 최고이고 5 개까지 단조 감소한다. 실제라우터를 쓰면 임베딩 라우팅 (4.94, 5.52) 이 단일 LoRA 기준선 (5.24, 5.65) 보다 **낮다**. 그리고 논문 자신의 벽시계 비교에서 RAG 가 논문이 제안한 방식보다 빠르다 —43.5 초 대 65.9 초. 그 비교에는 **학습 향이 아예 없다**.

시스템 모델링 관점 두 개의 knob 이 어긋난다. rank 는 **용량 knob** 이자 **바이트 knob** 인데, 절대 용량은 r 과 함께 오르고 파라미터당 효율은 작은 r 에서 최대다. 사용자별 저장소를 바이트로 사이징해야 하는 쪽은 작은 r 을 택하게된다. delta 트래픽이 실리는 자리도 정해져 있다 — 모든 설정에서 FFN-only LoRA 가 attn-only 를 일관되게 능가한다. 그리고 질의마다 병합이 일어나므로 **요청마다 가중치가 달라진다**. 두 요청이 서로 다른 delta 로 라우팅되면 같은 GEMM 을 공유할 수 없다. 독자가 아는 어휘로 옮기면 “batch 로 weight 공유” 라는 전제가 여기서 실제로 깨진다.

자세한 것은 ch09 § 09.4.

8. 용량과 계산은 서로 다른축이다

Θ 의 용량을 늘리는 방법이 “모델을 크게 만든다” 하나뿐이라면 용량논의는 곧 FLOPs 논의로 환원된다. Memory Layers at Scale(arXiv:2412.09764) 이그 환원을 끊는다.

기제는 FFN 층 하나를 학습 가능한 key-value lookup 으로 교체하는 것이다. 토큰마다 수백만 행 중 상위 몇 개만 골라 읽고 가중합한다. 결정적인 것은 그 key 와 value 가 **activation** 이 아니라 **학습된 파라미터**라는 점이다 — KV cache 가 아니고 E 도 아니다. Θ 안이다.

표 E04-3. memory layer 의 용량-FLOPs 분리 [memory-layers Table 1; 1.3b base, 1T tokens].

설정	총 파라미터	NaturalQuestions	TriviaQA
dense 1.3b	1.3b	7.76	32.64
Memory+ 1m values	3.377b	13.68	42.89
Memory+ 4m values	9.823b	14.43	51.18
Memory+ 16m values	35.618b	20.14	58.67
Memory+ 64m values	138.748b	20.78	62.14

총 파라미터가 **106.7 배**로 늘었는데 FLOPs 는 사실상 그대로다. 토큰마다 몇 행만 읽히기 때문이다. 그런데도 NQ 는 7.76 에서 20.78 로 오른다. 이것이 corpus 전체에서 용량 축과 계산 축이 **직교할 수 있다**는 것을 보이는 유일한 실증이다.

시스템 모델링 관점 그 대가가 어디로 갔는지는 분명하다. 이 층은 FLOP-bound 가 아니라 **bandwidth-bound** 다 — 논문의 forward 커널은 3 TB/s 에 도달하며 이는 H100 사양 3.35 TB/s 의 89.6%다. 그리고 dense GEMM 과 달리 **배치가 gather** 를 **상각하지 못한다**. 토큰마다 다른 행을 건드리므로 접근하는 행수가 배치 크기와 함께 늘어난다. 최대 설정의 memory 파라미터를 bf16 에 두면 274.9 GB 로 H100 한 장에 들어가지 않으며, 그래서 샤딩이 선택이 아니라 필수다 — 그러면 토큰 하나를 읽는 데 두 번의 collective 가임계경로에 들어오고, 이 왕복이 memory 층마다 반복된다. 마지막으로 이파라미터는 전 사용자 공유이므로 shared-weight batching 이 유지된다. 대가는 대칭적이다 — **사용자별 용량이 0** 이다. 개인 지식을 넣을자리가 이 설계에는 없다.

한계 정직성 항목이 셋이다. 첫째, 16m 에서 64m 구간은 파라미터를 103b 더 넣고 도 NQ 가 +0.64 점만 오른다. 논문은 이 구간을 “예측가능하게 계속 오른다” 고 쓰고 Llama2 7B 에 “근접한다” 고 쓰지만, 보고된 다섯 열 전부에서 뒤진다. 둘째, **dtype 이 논문 어디에도없고** 토큰마다 몇 행을 읽는지도 없다 — 위 문단의 274.9 GB 는 bf16 을 가정한 이 책의 조건부 환산이지 논문 수치가 아니다. 추론지연 · 처리량 측정은 0 건이다. 셋째, **이 논문은 sleep-time compute 의 사례가 아니다**. 가중치가 움직이는 시점은 pre-training 한 번뿐이고 배포 후 갱신이 없다. 여기서는 “그것이 쓰일그릇의 용량 실측치” 로만 인용된다.

자세한 것은 ch09 § 09.5.

9. 그릇이 모자라면 무엇이 일어나는가

지금까지는 그릇의 크기를 잴다. 그릇이 부족할 때 무슨 일이 생기는지는다른 질문이고, 답이 놀랍도록 깔끔하다.

과제를 맞히려면 반드시 있어야 하는 정보량이 있다. 상태 예산이 그것보다작으면, **오류율에 0 이 아닌 바닥이 생긴다**. 부족한 bits 에 비례해그 바닥이 올라간다. 이것은 Fano 부등식이라는 고전 결과를 압축 단계에적용해 나오며, 유도는 본문에 있다.

핵심 이 하한의 적용 범위가 이 절이 여기 있는 이유다 — 논문의 문장으로 “KV cache 든, gist 벡터든, recurrent state 든, 에이전트의 노트 저장소든, **어떤 압축 기억에 대해서도 성립한다**”. 상태를 Θ 에 쓰든 W 에 접든 E 에 적든 같은 형태의 바닥을 맞는다. **아키텍처로빠져나갈 구멍이 없다**.

sleep-time compute 에 직접 걸리는 항도 하나 나온다. **질의를 모르는채 압축하면 예산을 질의 분포 전체에 퍼야 한다**. 그만큼 유효 예산이 줄고, 그 차가 질의를 모르는 값의 정확한 가격이다. 그런데 sleep-time compute 은정의상 **질의가 도착하기 전에** 계산한다. 그러므로 세 경로 전부가구조적으로 이 가격을 낸다.

빠져나갈 유일한 문은 **되돌릴 수 있는가**다. 버린 것을 나중에 다시끌어올 수 있으면 질의가 온 뒤에 고를 수 있다. 실측이 그 차이를 잰다 —에이전트가 긴 문서를 읽으며 사실을 모으는 설정에서, 되돌릴 수 있는연산 (원본을 보관하고 질의 시 검색) 은 모든 압축 빈도에서 recall 0.95 근처를 유지하고, 되돌릴 수 없는 연산 (요약으로뒹어쓰기) 은 0.33–0.56 에 머문다.

한계 이 논문은 스스로 survey 이며 어느 쪽도 대규모로 실증하지 않는다고 쓴다. 실험은 1.5B 모델 하나를 RTX 4060 8GB 한 장에서 돌린 것이다. 더 무거운제한은 부등식 자체에 있다 — 유일한 자유 변수인 “과제가 요구하는정보량” 이 **한 번도 측정되지 않는다**. 논문 자신이 그 추정을 열린문제라고 적는다. 따라서 이 하한은 어느 설계 결정도 수치로 판정하지못한다. **형식은 쓸 수 있고 수치는 쓸 수 없다**.

자세한 것은 ch09 § 09.6.

10. 바이트로 옮기면 — 정보 밀도는 dtype 이정한다

지금까지의 수는 전부 “파라미터 몇 개”, “토큰 몇 개” 였다. 서빙 설계는바이트를 요구한다. 그런데 이 corpus 전체에서 상태 용량을바이트로 보고한 논문이 0 편이다. 그래서 이 책은 구성 파라미터에서바이트를 복원하는 결정론적 회계를 돌렸다. 계산은 암산으로 따라갈 수있다.

8B 급 앵커를 잡는다 — 폭 4096, 층 32 개, $\{q, v\}$ 두 행렬에 $r = 16$ LoRA. 행렬 하나에 붙는 delta 는 얇은 두 행렬이므로 $2 \times 16 \times 4096 = 131,072$ 개, 층마다 두 행렬이니 262,144 개, 32 층이니 총 8,388,608 개다. 8.39M 파라미터.

여기에 3.64 를 곱하면 약 3.05×10^7 bits, 즉 3.82 MB 다. 이 곱에 반드시 붙일 표기는 § 5 의 두 caveat 이다 — 3.64 는 바닥이고, 저랭크 부분공간에서는재측정된 적이 없다. 그러므로 이 값은 “이 delta 가 담는 양” 도 “넘을 수없는 양” 도 아니고 파라미터 수가 시사하는 정보 규모일 뿐이다.

이제 저장 바이트로 나눈다. 여기서부터가 방향이 보장되는 구간이다.

표 E04-4. 저장 형식별 정보 밀도 [X2 Q2]. Θ 행의 값은 전부하한이다.

저장 형식	저장 크기	bits / 저장 바이트
Θ delta, bf16	16.78 MB	1.82
Θ delta, int8	8.39 MB	3.64
Θ delta, int4	4.19 MB	7.28
E 텍스트 (액면가)	56.0 KB	8.00
E 텍스트 (3:1 압축 가정)	—	2.67

이 표에서 단정할 수 있는 것은 위 세 행뿐이다. 아래 두 행과 위 세 행을나눠 순서를 매기는 것이 § 5 에서 철회한 그 비교다. 위 세 행 안에서만정해지는 문장은 이렇다 — Θ -경로가 바이트 비효율로보이는 것은 파라미터당 최소 3.64 bits 를 16 bits 짜리 그릇에담기 때문이지, 표현이 정보를 담지 못해서가 아니다. 표현 자체에서 최소 4.4 배의 저장 여유가 나온다.

핵심 그러므로 delta 양자화는 Θ -경로 배포의 선택 사항이아니라 전제다. 이 결론은 3.64 가 바닥이든 천장이든 성립한다— 밀도의 분모인 저장 dtype 만으로 정해지기 때문이다. 그런데 이 corpus 의어느 논문도 delta 를 양자화한 상태에서 지식 회수율을 측정하지 않았다. 이것이 이 책이 지목하는 미탐색 영역 하나다.

시스템 모델링 관점 규모로 옮기면 기회의 성격이 드러난다. 사용자 10^6 명 기준 웹스토리지는 $r=16$ bf16 에서 16.777 TB, int4 로 내리면 4.194 TB, $r=64$ bf16 이면 67.109 TB 다. 모델을 통째로 복제하는 직접 편집 방식이면 16000 TB 로 세 자릿수가 뛴다 [X2 Q3]. 반대로 delta 하나는 작다 — 같은 8B 모델의 공유가중치가 bf16 으로 약 16 GB 인데 사용자당 delta 는 16.78 MB, 그 0.105%다. 문제는 크기가 아니라개수와 위치다. 사용자 수에 선형으로 늘고, 요청 지연 안에 워게층에서 끌어와 병합해야 하며, 병합된 순간 그 요청은 다른 사용자와 GEMM 을 공유하지 못한다. 즉 이것은 HBM 대역폭 문제가 아니라 워게층의 지연과 그 계층의 쓰기 내구성 문제다. Neural Memory 판이 W -경로에서 짙은 HBM read-modify-write 기회와는 다른 자리이며, 두결론을 하나로 잊지 않는다.

자세한 것은 ch09 § 09.7 과 ch09 Worked micro-example.

요약

- 이 권의 용량 단위는 bits 다. 재기 전에 “외운 것” 과 “아는 것” 을 참조모델로 갈라야 하고, 그 정의상 암기량은 모델 하나가 아니라 **모델쌍의 성질**이다.
- 3.64 bits per parameter 는 **균등·독립무작위 토큰 문자열을 GPT-2 로 from-scratch 학습시킨, 6.86×10^6 파라미터 이하 모델에서 나왔다.** 텍스트에 대한 값도 adapter 에 대한 값도 논문에 없다. 게다가 논문 안에서 여섯 가지로 인쇄되는 밴드이며, 어떤그림은 단위를 잘못 붙여 8 배를 과다 산정한다.
- 그 수는 천장이 아니라 **바닥**이다. 저자 자신이 “우리는 언제나 모델 용량의 하한만을 측정한다” 고 쓴다. 하한은 Θ -경로를반증하지 못한다 — “적어도 이만큼 들어간다” 는 그 경로에 유리한진술이다.
- 이 책의 회계 초판은 그것을 천장으로 놓고 경로 간 밀도 비를 냈다가**철회했다**. 하한과 액면가를 나눈 비는 어느 방향으로도 결론을주지 않는다. 살아남은 것은 dtype 축의 결론뿐이다.
- 용량은 (아키텍처, 학습 알고리즘) **쌍**의 성질이다. 정밀도를 두배로 올려도 용량은 9.1%만 오른다 — bpp 는 그릇의 성질이아니라 그 그릇을 채우는 절차의 성질이다.
- 방향이 보장된 유일한 천장은 “가중치 자체의 정보량” 이고, 실무형은 “찰때까지 채우고 차면 멈춘다” 이다. 포화 후의 쓰기는 제로섬이므로 반복되는 Θ -경로 라운드는 **반드시 잇는다**. 다만 이 논문은필연성만 주고 망각 속도 ρ 는 주지 않는다.
- LoRA delta 의 용량은 rank 와 함께 커지되 파라미터당 효율은 작은 rank 에서최대이고, 모듈을 여러 개 병합해 용량을 키우는 축은 **부정결과**다 — 모듈 1 개가 가장 좋고 5 개까지 단조 악화하며, 실제 라우터는단일 LoRA 기준선보다 낮다.
- 용량과 FLOPs 는 직교할 수 있다. 총 파라미터를 106.7 배로 늘리고 FLOPs 를사실상 고정한 채 NQ 7.76 $\rightarrow$ 20.78 을 얻은 실측이있다. 단 그 층은 FLOP-bound 가 아니라 bandwidth-bound 이고, **사용자별 용량은 0** 이다.
- 상태가 과제의 정보 요구량보다 작으면 $\Theta \cdot W \cdot E$ 어디에 두든 같은형태의 오류율 바닥을 맞고, 질의가 도착하기 전에 압축하는 모든 방식은질의를 모르는 값을 구조적으로 낸다.
- 저장 바이트당 정보 밀도는 표현의 한계가 아니라 **dtype 이정한다**: bf16 1.82 / int8 3.64 / int4 7.28. delta 양자화는최적화가 아니라 전제이며, 양자화 상태에서 회수율을 켜 논문은 아직없다.

자가 점검

- 3.64 bits per parameter 를 인용할 때 데이터 (무작위 균등문자열)·아키텍처 (GPT-2 from scratch)·규모 (6.86×10^6 파라미터 이하) 세 조건을함께 말할 수 있다.
- 그 값이 하한이라는 것이 무슨 뜻인지 **측정 절차**로 설명할 수있고, 천장으로 읽는 순간 어느 결론이 뒤집히는지 말할 수 있다.
- 표 E04-2 의 오른쪽 열 네 문장이 왜 쓸 수 없는 문장인지 각각 한 줄로 딸수 있다.
- 정밀도를 두 배로 올려도 용량이 9.1%만 오른다는 사실에서 “bpp 는 그릇의 성질이 아니다” 가 왜 따라 나오는지 설명할 수있다.
- 주어진 폭·층 수·rank·적용 행렬집합에서 LoRA delta 의 파라미터 수와 bf16/int4 저장바이트를 종이 없이 계산할 수 있다.

- memory layer 가 용량-FLOPs 분리의 실증이면서 동시에 sleep-time compute 이 아닌 이유를 말할 수 있다.
- (Rosetta) “checkpoint 크기 $\leftrightarrow$ 상태 용량” 행을 수로 채울 수 있다 — 8B bf16 checkpoint 약 16 GB, 그 안의 $r=16$ LoRA delta 16.78 MB(0.105%), 그 delta 가 시사하는 정보규모 3.82 MB. 세 수가 왜 전부 다른 양인지 설명할 수 있다.

다음 권으로

이 권이 세운 것은 Θ 의 천장이다. 그 천장은 **가중치에만** 걸린다. 모델 밖에 둔 바이트에는 걸리지 않는다.

그렇다면 자연스러운 다음 질문이 나온다 — 상태를 E 로 옮기면 이 천장을 피할 수 있는가. 답은 절반만 그렇다. 그릇 상한은 사라지지만 §9의 오류율바닥은 그대로 남고, E 는 대신 **질의마다** 검색 지연과 프롬프트바이트를 낸다. 다음 권은 그 E 의 비용 구조 — 무엇이 E 의 용량을 제한하고, 읽기 한 번에 무엇이 얼마나 움직이는가 — 를 받는다.